

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,

Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

The complete source code, datasets, generated plots, and report files for this experiment are available in the following GitHub repository:

GitHub Repository:

<https://github.com/VIKASNI-S/Deep-Learning-Lab/tree/main/Lab-3>

10. Mandatory Plots

Generate the following plots:

1. Sample Images



Figure 1: Sample images from the CIFAR-10 dataset representing its ten object classes.

Sample images illustrate the diversity of objects present in the CIFAR-10 dataset across its ten classes.

2. Class Distribution

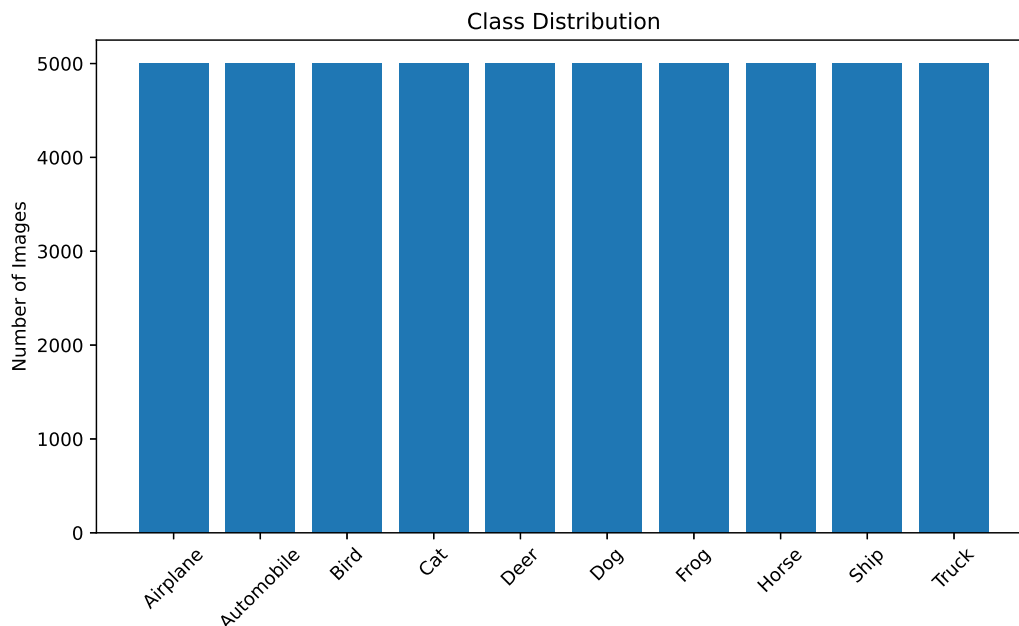


Figure 2: Distribution of images across the ten classes in the CIFAR-10 training dataset.

Dataset contains equal no. of images of all 10 classes – i , balanced dataset. This prevents the model from being biased towards a particular class.

3. Training Accuracy and Validation Accuracy

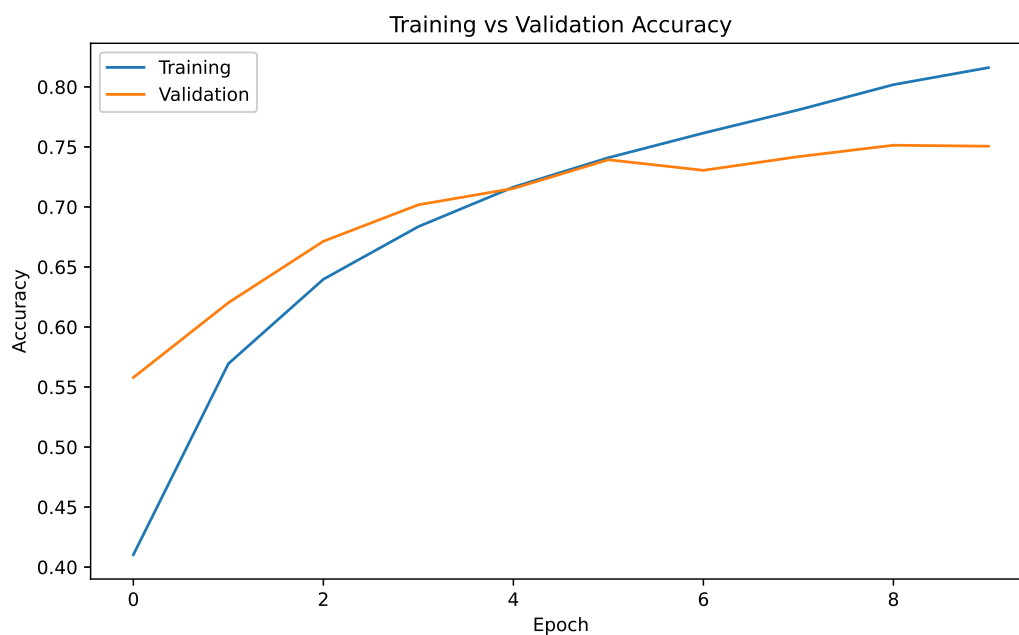


Figure 3: Training and validation accuracy over training epochs.

Training and Validation accuracy increase steadily with epochs, demonstrating that the CNN learns meaningful image features. The small gap depicts good generalization with minimal overfitting.

4. Training Loss and Validation Loss



Figure 4: Training and validation loss over training epochs.

The loss decreases consistently during training, showing successful optimization of the network parameters. The validation loss follows a similar trend, indicating stable learning and good convergence.

5. Feature Maps

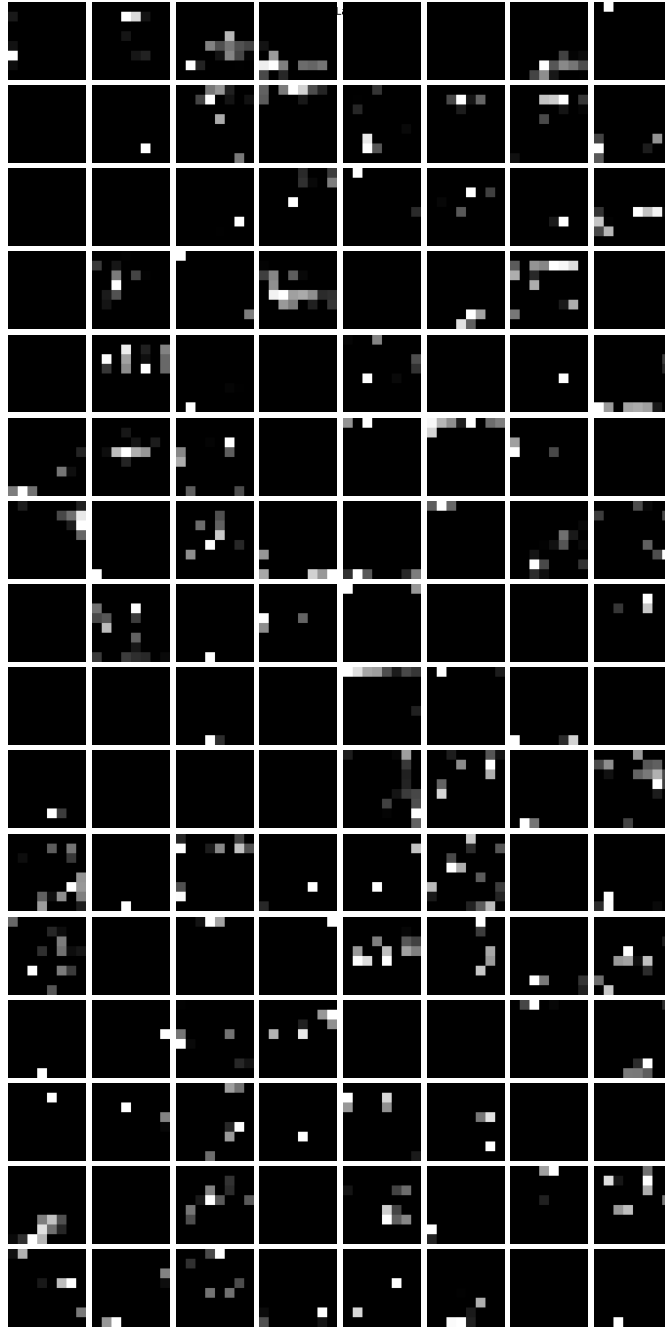


Figure 5: Feature maps generated by the third convolutional layer.

3rd Convolution layer learns high level semantic features like object shapes and class specific patterns. Also suppresses irrelevant background information.

6. Confusion Matrix

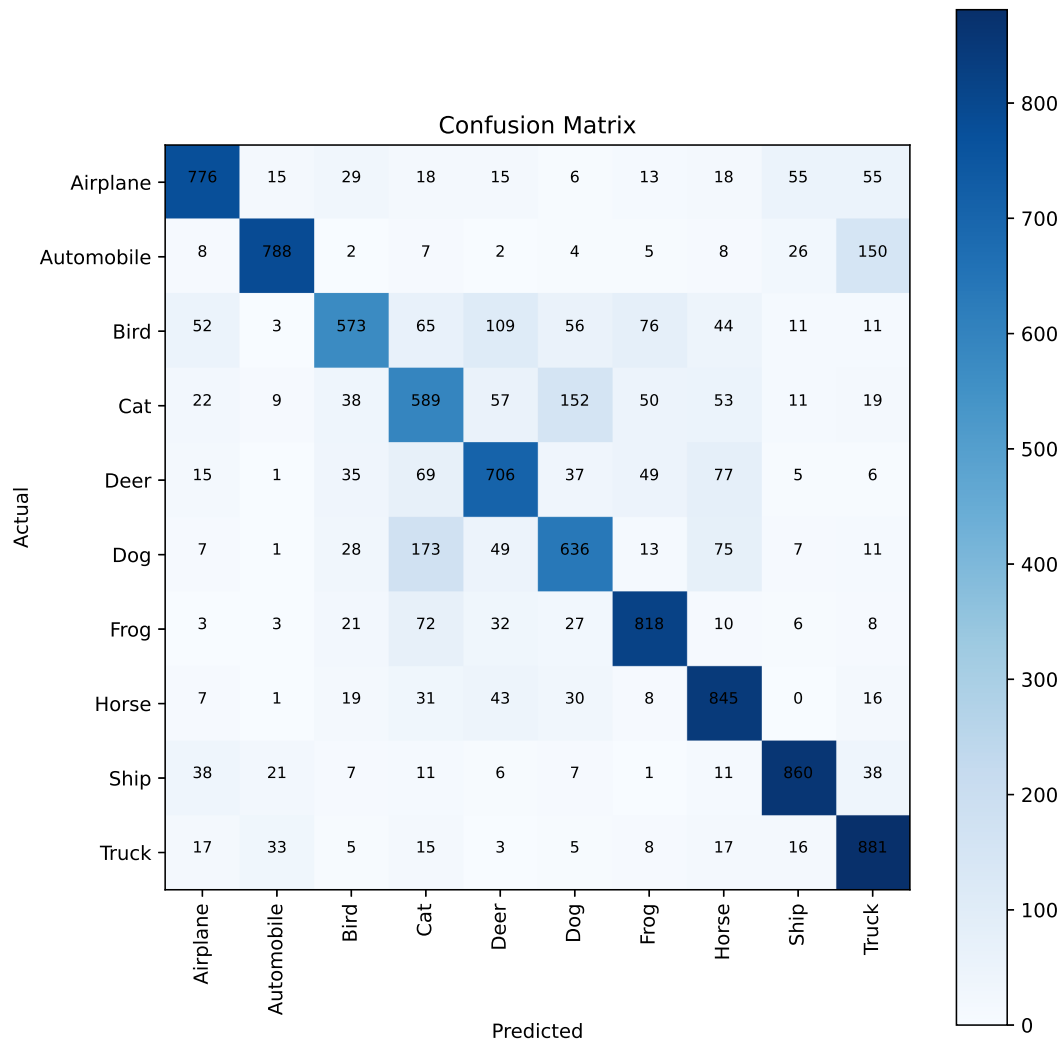


Figure 6: Confusion matrix of the CNN model on the CIFAR-10 test dataset.

Most predictions lie along the diagonal showing that the model correctly classifies the majority of images. There is chance for misclassification between visually similar classes.

12. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

0.1 Evaluation Metrics

Table 1: Performance Evaluation Metrics of the CNN Model

Metric	Value
Accuracy	0.7472
Precision	0.7508
Recall	0.7472
F1-Score	0.7461

0.2 8. Classification Report

Table 2: Classification Report of the CNN Model on the CIFAR-10 Test Dataset

Class	Precision	Recall	F1-Score	Support
Airplane	0.82	0.78	0.80	1000
Automobile	0.90	0.79	0.84	1000
Bird	0.76	0.57	0.65	1000
Cat	0.56	0.59	0.57	1000
Deer	0.69	0.71	0.70	1000
Dog	0.66	0.64	0.65	1000
Frog	0.79	0.82	0.80	1000
Horse	0.73	0.84	0.78	1000
Ship	0.86	0.86	0.86	1000
Truck	0.74	0.88	0.80	1000
Accuracy	0.75			10000
Macro Average	0.75	0.75	0.75	10000
Weighted Average	0.75	0.75	0.75	10000

0.3 CNN Model Architecture Summary

Table 3: Summary of the CNN Architecture

Layer (Type)	Output Shape	Parameters
InputLayer	(None, 32, 32, 3)	0
Conv2D	(None, 30, 30, 8)	224
MaxPooling2D	(None, 15, 15, 8)	0
Flatten	(None, 1800)	0
Dense	(None, 10)	18,010

Total Parameters	18,234
Trainable Parameters	18,234
Non-trainable Parameters	0

““latex

13. Discussion

- Why is convolution preferred over fully connected layers for images?**
Convolution preserves the spatial structure of images and detects local patterns such as edges and textures. It also uses shared weights, reducing the number of parameters.
- How does stride affect the feature map size?**
A larger stride moves the filter by more pixels, producing a smaller feature map. A smaller stride gives a larger feature map with more spatial information.
- What is the role of padding?**
Padding adds extra pixels, usually zeros, around the image boundaries. It helps preserve spatial dimensions and allows edge features to be detected effectively.
- Why is pooling used?**
Pooling reduces the spatial dimensions of feature maps, decreasing computation and parameters. It also provides some translation invariance and helps reduce overfitting.
- How do feature maps represent image characteristics?**
Feature maps show the locations where specific features detected by convolutional filters are present. Early layers detect edges and textures, while deeper layers detect complex shapes and objects.
- Why do CNNs require fewer parameters than MLPs?**
CNNs use local connectivity and weight sharing, so the same filter is applied across different image regions. This greatly reduces the number of trainable parameters compared with fully connected MLPs.

““

14. References

- Goodfellow et al., *Deep Learning*.
- Bishop, *Pattern Recognition and Machine Learning*.
- Haykin, *Neural Networks and Learning Machines*.
- TensorFlow Documentation.
- CIFAR-10 Dataset Documentation.